

Laboratory Guide 4: Variational Quantum Algorithms

Hybrid Classical-Quantum Computing: VQE and QAOA

Course: Quantum Computing Foundation

Contents

1	Introduction and Objectives	2
2	Part A: The Hybrid Paradigm	3
2.1	Architecture of the Variational Loop	3
2.2	The Ansatz: Expressibility vs. Trainability	3
3	Part B: The Parameter-Shift Rule	3
3.1	The Gradient Problem in Quantum Circuits	3
3.2	Derivation	4
3.3	Concrete Example	5
3.4	Barren Plateaus	6
4	Part C: VQE — The Variational Quantum Eigensolver	6
4.1	Goal and Formulation	6
4.2	Case Study: 2-Qubit Ising Model with a Longitudinal Field	6
4.3	Hardware-Efficient Ansatz for H	6
4.4	Gradient Computation via Parameter-Shift	7
5	Part D: QAOA — Quantum Approximate Optimization	7
5.1	From VQE to Combinatorial Optimization	7
5.2	Case Study: MaxCut on a 4-Node Cycle	7
5.3	Approximation Ratio and the Role of p	8
6	Part E: IBM Composer Exercises	9
6.1	Exercise 1: The Parameter-Shift Rule in Practice	9
6.2	Exercise 2: Visualise the Cost Landscape	10
7	Part F: Qiskit Python Implementation	10
8	Part G: Bridge to Lab 4 Finance and Assessment	11
8.1	Assessment	11

1 Introduction and Objectives

Labs 3, 3.5 and 3.6 introduced the foundations of quantum interference, search, and integer factoring. All those algorithms assumed a *fault-tolerant* quantum computer: a device with essentially unlimited coherence time and negligible gate error. Today's hardware — the so-called NISQ era — does not meet this assumption. Circuits deeper than a few dozen layers are dominated by noise before they can return a useful result.

This laboratory introduces the practical answer to the NISQ limitation: **variational quantum algorithms**, also called *hybrid classical-quantum* algorithms. Rather than running a single deep quantum circuit, these algorithms iterate between a short parametric quantum circuit and a classical optimizer, converging toward a useful solution. Understanding this paradigm is a prerequisite for Lab 4 Finance, where you will apply it to a real-world portfolio optimization problem.

Prerequisites:

- Labs 1–2 (gates, CNOT, two-qubit states)
- Basic linear algebra: eigenvalues, Hermitian matrices, tensor products
- Classical optimization concepts: gradient descent, cost functions

Learning Objectives:

1. Understand why the NISQ era requires a hybrid paradigm.
2. Master parametric quantum circuits (PQC) and the concept of ansatz.
3. Derive and apply the *parameter-shift rule* for quantum gradients.
4. Implement the Variational Quantum Eigensolver (VQE) for a 2-qubit Ising model.
5. Implement QAOA for MaxCut and understand the QUBO framework that bridges directly to financial optimization.

The NISQ Bottleneck

Current quantum computers (IBM Eagle, Heron; Google Willow; IonQ Forte) have:

- **Gate errors:** $\sim 0.1\text{--}1\%$ per two-qubit gate.
- **Coherence times:** T_1, T_2 of tens to a few hundred μs on superconducting chips (IBM, Google); on trapped ions (IonQ) T_2 is about one second and T_1 even longer, but the gates are also much slower.
- **Depth limit:** without error mitigation, circuits with more than a few hundred two-qubit gates produce results dominated by noise; with error mitigation, IBM has run circuits of about 3,000 (2023) to 5,000 (2024) two-qubit gates.

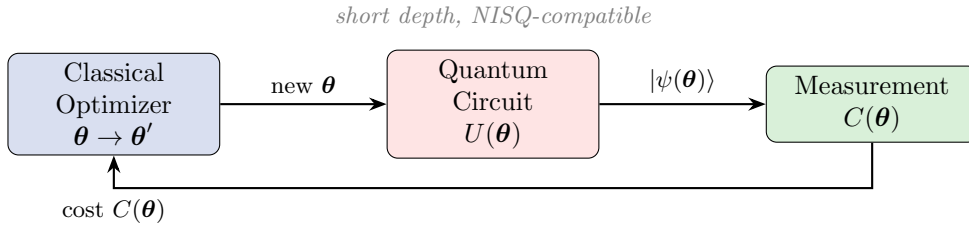
Shor's Algorithm for RSA-2048 requires billions of fault-tolerant gates on about 1,400 error-corrected (logical) qubits: just under a million noisy physical qubits in a 2025 estimate (Gidney). VQE and QAOA are designed to run on circuits with $\mathcal{O}(n)$ – $\mathcal{O}(n^2)$ gates and can *tolerate some noise* because the classical optimizer can compensate for small systematic errors.

2 Part A: The Hybrid Paradigm

2.1 Architecture of the Variational Loop

A variational quantum algorithm operates as follows:

1. **Encode:** Map the problem into a Hamiltonian H such that the solution corresponds to the ground state (lowest eigenvalue) of H .
2. **Ansatz:** Choose a parametric quantum circuit $U(\boldsymbol{\theta})$ — the *ansatz* — that produces the trial state $|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})|0\rangle$.
3. **Measure:** Evaluate the cost function $C(\boldsymbol{\theta}) = \langle\psi(\boldsymbol{\theta})|H|\psi(\boldsymbol{\theta})\rangle = \langle H \rangle_{\boldsymbol{\theta}}$.
4. **Optimise:** A *classical* optimizer updates $\boldsymbol{\theta} \rightarrow \boldsymbol{\theta}'$ to reduce C .
5. **Repeat:** Steps 3–4 iterate until convergence.



2.2 The Ansatz: Expressibility vs. Trainability

The ansatz $U(\boldsymbol{\theta})$ must satisfy two competing requirements:

- **Expressibility:** the set of states $\{|\psi(\boldsymbol{\theta})\rangle\}$ should include (or approximate) the true ground state.
- **Trainability:** gradients $\partial C / \partial \theta_k$ must be non-negligible so the optimizer can navigate the landscape.

Hardware-Efficient Ansatz (HEA)

An ansatz built from gates native to the target hardware: alternating layers of single-qubit rotations $R_y(\theta)$, $R_z(\phi)$ and entangling CNOT gates. For n qubits and L layers, the circuit has $\mathcal{O}(nL)$ parameters. Maximises expressibility within the coherence budget, but may not respect problem symmetry.

3 Part B: The Parameter-Shift Rule

3.1 The Gradient Problem in Quantum Circuits

Classical neural networks use backpropagation to compute gradients exactly in one backward pass. In a quantum circuit, however, we cannot backpropagate through a unitary — we can only *evaluate* the circuit at different parameter settings. The **parameter-shift rule** (Mitarai et al., 2018; Schuld et al., 2019) gives us an exact gradient using only two additional circuit evaluations.

3.2 Derivation

Consider a gate of the form $G(\theta) = e^{-i\theta P/2}$, where P is a Hermitian operator with eigenvalues ± 1 (a Pauli operator: $P \in \{X, Y, Z, X \otimes Z, \dots\}$). Then:

$$G(\theta) = e^{-i\theta P/2} = \cos \frac{\theta}{2} I - i \sin \frac{\theta}{2} P. \quad (1)$$

The cost function (measuring observable O after applying a circuit $V(\theta) = \dots G(\theta) \dots$ to state ρ) is:

$$f(\theta) = \text{Tr} \left[O V(\theta) \rho V^\dagger(\theta) \right]. \quad (2)$$

Parameter-Shift Rule — Full Derivation

Step 1: Expand $G(\theta)$. Using $\cos(\theta/2) = \frac{e^{i\theta/2} + e^{-i\theta/2}}{2}$ and $\sin(\theta/2) = \frac{e^{i\theta/2} - e^{-i\theta/2}}{2i}$, and denoting the context of the circuit (all gates before and after G) as a fixed superoperator $\mathcal{L}$, we can write:

$$f(\theta) = \text{Tr}[\tilde{O}(\theta)] = a + b \cos \theta + c \sin \theta \quad (3)$$

where $a, b, c \in \mathbb{R}$ depend on O, ρ and the rest of the circuit, but *not* on θ . This is proved by expanding:

$$\begin{aligned} V(\theta)\rho V^\dagger(\theta) &= \left(\cos \frac{\theta}{2} I - i \sin \frac{\theta}{2} P\right) \tilde{\rho} \left(\cos \frac{\theta}{2} I + i \sin \frac{\theta}{2} P\right) \\ &= \cos^2 \frac{\theta}{2} \tilde{\rho} + \sin^2 \frac{\theta}{2} P \tilde{\rho} P + i \sin \frac{\theta}{2} \cos \frac{\theta}{2} [\tilde{\rho}, P], \end{aligned}$$

where $\tilde{\rho}$ absorbs all gates not containing θ . Applying $\text{Tr}[O \cdot]$ and using $\cos^2 = (1 + \cos \theta)/2$, $\sin^2 = (1 - \cos \theta)/2$, $\sin(\theta/2) \cos(\theta/2) = \sin \theta/2$:

$$\begin{aligned} f(\theta) &= \frac{1 + \cos \theta}{2} \text{Tr}[O \tilde{\rho}] + \frac{1 - \cos \theta}{2} \text{Tr}[OP \tilde{\rho} P] + \frac{\sin \theta}{2} \underbrace{i \text{Tr}[O[\tilde{\rho}, P]]}_{\in \mathbb{R}} \\ &= \underbrace{\frac{1}{2}(\text{Tr}[O \tilde{\rho}] + \text{Tr}[OP \tilde{\rho} P])}_a + \underbrace{\frac{1}{2}(\text{Tr}[O \tilde{\rho}] - \text{Tr}[OP \tilde{\rho} P])}_b \cos \theta + c \sin \theta. \end{aligned}$$

Step 2: Differentiate the trigonometric form.

Since $f(\theta) = a + b \cos \theta + c \sin \theta$:

$$\frac{df}{d\theta} = -b \sin \theta + c \cos \theta. \quad (4)$$

Step 3: Express the derivative via two circuit evaluations.

Evaluate f at the shifted points:

$$f\left(\theta + \frac{\pi}{2}\right) = a + b \cos\left(\theta + \frac{\pi}{2}\right) + c \sin\left(\theta + \frac{\pi}{2}\right) = a - b \sin \theta + c \cos \theta, \quad (5)$$

$$f\left(\theta - \frac{\pi}{2}\right) = a + b \cos\left(\theta - \frac{\pi}{2}\right) + c \sin\left(\theta - \frac{\pi}{2}\right) = a + b \sin \theta - c \cos \theta. \quad (6)$$

Subtracting:

$$f\left(\theta + \frac{\pi}{2}\right) - f\left(\theta - \frac{\pi}{2}\right) = -2b \sin \theta + 2c \cos \theta = 2 \frac{df}{d\theta}. \quad (7)$$

Result:

$$\boxed{\frac{\partial f}{\partial \theta_k} = \frac{1}{2} \left[f\left(\theta_k + \frac{\pi}{2}\right) - f\left(\theta_k - \frac{\pi}{2}\right) \right]}. \quad (8)$$

This is **exact**, not an approximation. The gradient of any expectation value with respect to a Pauli-generator rotation gate requires exactly **two additional circuit evaluations**.

3.3 Concrete Example

For the circuit $R_y(\theta)|0\rangle$ measuring $\langle Z \rangle$:

$$f(\theta) = \langle 0 | R_y^\dagger(\theta) Z R_y(\theta) | 0 \rangle = \cos^2 \frac{\theta}{2} - \sin^2 \frac{\theta}{2} = \cos \theta.$$

Applying equation (8):

$$\frac{df}{d\theta} = \frac{f(\theta + \pi/2) - f(\theta - \pi/2)}{2} = \frac{\cos(\theta + \pi/2) - \cos(\theta - \pi/2)}{2} = \frac{-\sin \theta - \sin \theta}{2} = -\sin \theta. \quad \checkmark$$

3.4 Barren Plateaus

The parameter-shift rule guarantees we can *compute* gradients, but not that gradients are *large*. As the number of qubits n grows, random parametric circuits suffer the **barren plateau** phenomenon:

Barren Plateaus (McClean et al., 2018)

For a random n -qubit ansatz, the variance of the gradient $\partial C / \partial \theta_k$ decreases exponentially:

$$\text{Var} \left[\frac{\partial C}{\partial \theta_k} \right] = \mathcal{O} \left(\frac{1}{2^n} \right).$$

This means gradients are exponentially small in expectation — the optimizer is lost on a flat landscape. Mitigations include: problem-inspired ansätze, layerwise training, natural gradient methods, and classical pre-training.

4 Part C: VQE — The Variational Quantum Eigensolver

4.1 Goal and Formulation

VQE (Peruzzo et al., 2014) finds the ground-state energy of a Hamiltonian H by exploiting the *variational principle* of quantum mechanics:

$$E_0 \leq C(\boldsymbol{\theta}) = \langle \psi(\boldsymbol{\theta}) | H | \psi(\boldsymbol{\theta}) \rangle \quad \forall \boldsymbol{\theta}. \quad (9)$$

The optimizer minimises $C(\boldsymbol{\theta})$ over $\boldsymbol{\theta}$, pushing the trial state toward the ground state.

4.2 Case Study: 2-Qubit Ising Model with a Longitudinal Field

We work with the Hamiltonian:

$$H = Z_0 Z_1 + 0.5 Z_0, \quad (10)$$

where Z_i denotes the Pauli Z operator on qubit i . The $Z_0 Z_1$ term (coefficient +1) penalises states where the two qubits are aligned and favours anti-aligned ones; $0.5 Z_0$ introduces a longitudinal field on qubit 0.

Ground State of Equation (10)

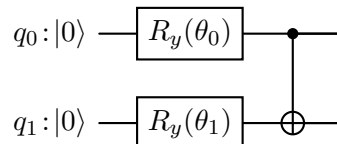
The matrix of H in the $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ basis is diagonal (since H is a sum of Pauli- Z products):

$$H = \text{diag}(1 + 0.5, -1 + 0.5, -1 - 0.5, 1 - 0.5) = \text{diag}(1.5, -0.5, -1.5, 0.5).$$

- Ground state: $|10\rangle$ with energy $E_0 = -1.5$.
- First excited: $|01\rangle$ with energy -0.5 .

4.3 Hardware-Efficient Ansatz for H

We use a 2-parameter ansatz:



$$|\psi(\theta_0, \theta_1)\rangle = \text{CX}_{01} (R_y(\theta_0) \otimes R_y(\theta_1))|00\rangle.$$

The cost function:

$$C(\theta_0, \theta_1) = \langle \psi(\theta_0, \theta_1) | (Z_0 Z_1 + 0.5 Z_0) | \psi(\theta_0, \theta_1) \rangle. \quad (11)$$

Student Activity

Verify: Set $\theta_0 = \pi$, $\theta_1 = \pi$. Show that $|\psi(\pi, \pi)\rangle = |10\rangle$ and that $C(\pi, \pi) = -1.5 = E_0$.
Hint: $R_y(\pi)|0\rangle = |1\rangle$ on both qubits gives $|11\rangle$, and CX (control $q_0 = 1$) flips the target: $|11\rangle \rightarrow |10\rangle$.

4.4 Gradient Computation via Parameter-Shift

For θ_0 :

$$\frac{\partial C}{\partial \theta_0} = \frac{1}{2} [C(\theta_0 + \frac{\pi}{2}, \theta_1) - C(\theta_0 - \frac{\pi}{2}, \theta_1)]. \quad (12)$$

A gradient-based optimizer uses these gradient estimates to descend toward the minimum (π, π) : the web page's VQE simulator takes plain gradient-descent steps; the notebook uses COBYLA, which needs no gradients.

5 Part D: QAOA — Quantum Approximate Optimization

5.1 From VQE to Combinatorial Optimization

VQE can be applied to *any* Hamiltonian whose ground state encodes the optimal solution to a problem. For *combinatorial* problems over binary variables $x \in \{0, 1\}^n$, the standard encoding uses:

$$z_i = 1 - 2x_i \in \{-1, +1\} \iff x_i = \frac{1 - Z_i}{2}. \quad (13)$$

Any quadratic binary objective function $\min_x \sum_{ij} Q_{ij} x_i x_j + \sum_i q_i x_i$ maps to a Hamiltonian that is a sum of Pauli-Z products.

QAOA (Farhi et al., 2014) is a VQE with a *structured* ansatz inspired by adiabatic quantum computing. It alternates two unitaries for p layers:

$$|\gamma, \beta\rangle = e^{-i\beta_p H_B} e^{-i\gamma_p H_C} \dots e^{-i\beta_1 H_B} e^{-i\gamma_1 H_C} |+\rangle^{\otimes n}, \quad (14)$$

where:

- H_C is the **cost Hamiltonian** encoding the problem.
- $H_B = \sum_i X_i$ is the **mixer Hamiltonian** promoting transitions between solutions.
- $(\gamma, \beta) \in \mathbb{R}^{2p}$ are optimised classically.

As $p \rightarrow \infty$, QAOA converges to the exact solution.

5.2 Case Study: MaxCut on a 4-Node Cycle

MaxCut

Given a graph $G = (V, E)$, partition V into two sets S and $\bar{S}$ to maximise the number of edges crossing the partition (i.e. edges with one endpoint in S and one in $\bar{S}$).

We use the 4-node cycle C_4 with edges $E = \{(0, 1), (1, 2), (2, 3), (3, 0)\}$ (see Figure 1).

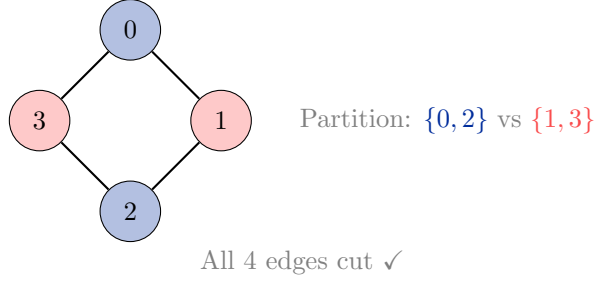


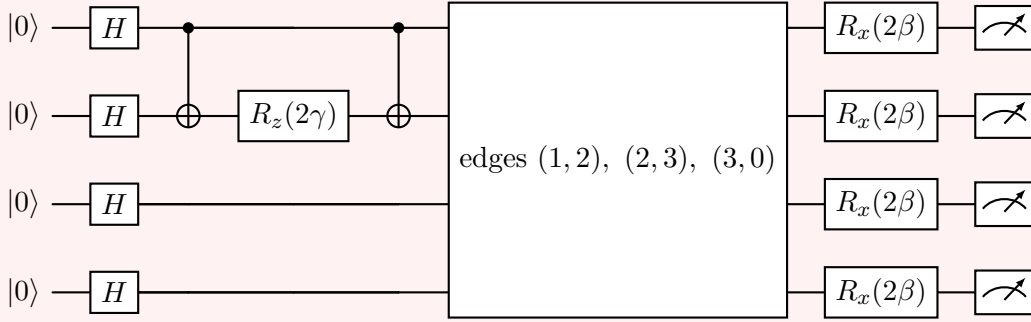
Figure 1: 4-node cycle C_4 with the optimal MaxCut partition.

The cost Hamiltonian encoding the number of cut edges:

$$\begin{aligned}
 H_C &= \frac{1}{2} \sum_{(i,j) \in E} (I - Z_i Z_j) \\
 &= 2I - \frac{1}{2} (Z_0 Z_1 + Z_1 Z_2 + Z_2 Z_3 + Z_3 Z_0).
 \end{aligned} \tag{15}$$

The maximum cut value is 4, achieved by $|0101\rangle$ and $|1010\rangle$ (alternating-spin states, corresponding to the partition $\{0, 2\}$ vs $\{1, 3\}$).

QAOA Circuit — $p = 1$, Single Layer



The cost layer applies, for each edge (i, j) : $\text{CNOT}(i, j) \text{ — } R_z(2\gamma) \text{ on } j \text{ — } \text{CNOT}(i, j)$. The figure draws this block for the edge $(0, 1)$; the box stands for the same block on the edges $(1, 2)$, $(2, 3)$ and $(3, 0)$. Each block implements $e^{-i\gamma Z_i Z_j}$:

$$e^{-i\gamma Z_i Z_j} = \text{CNOT}_{ij} \cdot (I \otimes e^{-i\gamma Z_j}) \cdot \text{CNOT}_{ij}.$$

Since $H_C = \sum_{(i,j) \in E} (I - Z_i Z_j)/2$, the four blocks together equal $e^{-i\gamma' H_C}$ with $\gamma' = -2\gamma$, times an overall phase factor that changes no probability: the same family of circuits with a rescaled angle (the convention of the web page and the notebook). The mixer $e^{-i\beta H_B} = \bigotimes_i R_x(2\beta)$ applies a global X -rotation to all qubits.

5.3 Approximation Ratio and the Role of p

Define the **approximation ratio**: $r = \langle H_C \rangle / \text{MaxCut}$. For C_4 with $p = 1$, the optimal ratio is $r \approx 0.75$ (3 expected cuts out of 4). Increasing p improves r :

p	Parameters	Approx. ratio (simulation)
1	2	≈ 0.75
2	4	1.00
3	6	1.00

(exact on C_4 from $p = 2$)

6 Part E: IBM Composer Exercises

Platform: IBM Quantum Composer — <https://quantum.ibm.com/composer>

6.1 Exercise 1: The Parameter-Shift Rule in Practice

Student Activity

Observe the parameter-shift rule directly:

1. Build $R_y(\theta)|0\rangle$ and measure $\langle Z \rangle$. This is $f(\theta) = \cos \theta$.
2. Evaluate at $\theta = 0.5$ rad. Expected: $f(0.5) = \cos(0.5) \approx 0.878$.
3. Change to $\theta = 0.5 + \pi/2 \approx 2.071$ rad. Record $f^+ \approx -0.479$.
4. Change to $\theta = 0.5 - \pi/2 \approx -1.071$ rad. Record $f^- \approx +0.479$.
5. PSR gradient = $(f^+ - f^-)/2 \approx -0.479$. Analytic: $-\sin(0.5) \approx -0.479$. The rule is exact: the measured values differ from the analytic ones only by shot noise.

OpenQASM 2.0 — three circuits to run in sequence:

```
1 OPENQASM 2.0;
2 include "qelib1.inc";
3 // Circuit 1: f(theta) at theta = 0.5 rad
4 qreg q[1]; creg c[1];
5 ry(0.5) q[0];
6 measure q[0] -> c[0];
7 // Expected <Z> = cos(0.5) approx. 0.878
8
9 // Circuit 2: f(theta + pi/2) -- positive shift
10 // Replace ry(0.5) with ry(2.0708)
11 // ry(2.0708) q[0]; // theta + pi/2 = 0.5 + 1.5708
12
13 // Circuit 3: f(theta - pi/2) -- negative shift
14 // Replace ry(0.5) with ry(-1.0708)
15 // ry(-1.0708) q[0]; // theta - pi/2 = 0.5 - 1.5708
16
17 // PSR gradient = (result_2 - result_3) / 2 approx. -0.479
18 // Analytic: -sin(0.5) approx. -0.4794 --> identical
```

Expected: The parameter-shift value equals the analytic gradient $-\sin(0.5) \approx -0.479$; any difference in your measured values comes only from shot noise.

6.2 Exercise 2: Visualise the Cost Landscape

Student Activity

Two-qubit VQE landscape:

1. Build the ansatz $CX_{01}(R_y(\theta_0) \otimes R_y(\theta_1))$.
2. Fix $\theta_1 = \pi$. Sweep $\theta_0 \in \{0, \pi/4, \pi/2, 3\pi/4, \pi\}$.
3. For each value, add $Z_0 Z_1$ and Z_0 measurement and record $C(\theta_0, \pi)$ from the Probabilities panel.
4. Plot by hand: along this line $C = -1 + \frac{1}{2} \cos \theta_0$, so you should see C decrease monotonically from -0.5 (at $\theta_0 = 0$) to -1.5 (at $\theta_0 = \pi$), the global minimum.

7 Part F: Qiskit Python Implementation

The full implementation is in the companion Jupyter notebook. Three exercises are provided:

1. **Parameter-shift demo:** compute the gradient of $\langle Z \rangle_\theta$ analytically and via the rule, verify agreement.
2. **VQE on Ising:** use Qiskit's Estimator primitive with COBYLA to minimise $C(\theta_0, \theta_1)$.
3. **QAOA MaxCut:** sweep $p \in \{1, 2, 3\}$ and compare approximation ratios and convergence speed.

Key Code Structure

```
1 # VQE cost function (Estimator-based)
2 def cost_function(params, estimator, circuit, hamiltonian):
3     job = estimator.run([(circuit, hamiltonian, params)])
4     result = job.result()
5     return result[0].data.evs # expectation value
6
7 # QAOA circuit for MaxCut (one p-layer)
8 def build_qaoa(n, edges, p):
9     gammas = [Parameter(f'g{i}') for i in range(p)]
10    betas = [Parameter(f'b{i}') for i in range(p)]
11    qc = QuantumCircuit(n)
12    qc.h(range(n))
13    for k in range(p):
14        for (u, v) in edges: # cost unitary
15            qc.cx(u, v)
16            qc.rz(2 * gammas[k], v)
17            qc.cx(u, v)
18        qc.rx(2 * betas[k], range(n)) # mixer
19    return qc, gammas + betas
```

8 Part G: Bridge to Lab 4 Finance and Assessment

Bridge to Lab 4 Finance: Portfolio Optimization

From MaxCut to Portfolio Optimization

In Lab 4 Finance, the portfolio optimization problem is formulated as:

$$\min_{x \in \{0,1\}^n} x^T Q x \quad \text{subject to} \quad \mathbf{1}^T x = K,$$

where $x_i = 1$ means "invest in asset i ", K is the target portfolio size, and Q_{ij} encodes the risk-return trade-off (covariance minus expected return).

This is a **QUBO** (Quadratic Unconstrained Binary Optimization) and maps to a Hamiltonian via $Z_i = 1 - 2x_i$:

$$H_{\text{portfolio}} = \sum_{i,j} Q_{ij} \frac{(I - Z_i)(I - Z_j)}{4} + \lambda \left(\sum_i \frac{I - Z_i}{2} - K \right)^2.$$

The second term is a *penalty* enforcing the budget constraint. QAOA is then applied to $H_{\text{portfolio}}$ using *exactly the same circuit structure* built in this lab, but with this financial Hamiltonian as H_C .

The skills you need: variational loop (§2.1), parameter-shift gradients (§3), QAOA circuit construction (§5).

8.1 Assessment

Q1. Parameter-Shift Rule

Why does the parameter-shift rule require *exactly two* additional circuit evaluations per parameter, regardless of the circuit depth?

- A) Because the gradient is approximated by a finite difference.
- B) Because $f(\theta)$ is a trigonometric polynomial of degree 1 in θ (for Pauli-generator gates), and its derivative is uniquely determined by evaluations at $\theta \pm \pi/2$.
- C) Because quantum circuits have only one parameter each.

Q2. Barren Plateaus

Which strategy does *not* mitigate the barren plateau problem?

- A) Using problem-inspired ansätze that respect problem symmetry.
- B) Increasing the number of qubits to improve expressibility.
- C) Layerwise training, initialising one layer at a time.

Q3. QAOA Structure

In QAOA, what is the role of the mixer Hamiltonian $H_B = \sum_i X_i$?

- A) To encode the cost function directly into the circuit.
- B) To create transitions between different bitstring states, allowing the optimizer to escape local minima.
- C) To initialise all qubits in the computational basis.

Q4. QUBO Connection

A portfolio with $n = 4$ assets uses a QUBO objective $x^T Q x$. After substituting $x_i = (1 - Z_i)/2$, the resulting Hamiltonian H_C contains terms of the form $Z_i Z_j$. What gate in the QAOA circuit directly implements $e^{-i\gamma Z_i Z_j}$?

- A) A controlled-Z gate.
- B) CNOT — $R_z(2\gamma)$ — CNOT on qubits i and j .
- C) Two $R_z(2\gamma)$ gates applied independently.

Solutions: Q1: B (the trigonometric form $a + b \cos \theta + c \sin \theta$ is fundamental — any expectation value w.r.t. a Pauli generator is of this form, making the two-point shift exact, not approximate); Q2: B (more qubits make barren plateaus *worse*, not better); Q3: B (the mixer drives transitions, analogous to kinetic energy in the adiabatic picture — without it the circuit is stuck in the product state and cannot explore the solution space); Q4: B (the standard ZZ-rotation decomposition: $e^{-i\gamma Z_i Z_j} = \text{CNOT}_{ij} \cdot (I \otimes e^{-i\gamma Z_j}) \cdot \text{CNOT}_{ij}$, with $e^{-i\gamma Z} = R_z(2\gamma)$ up to global phase).